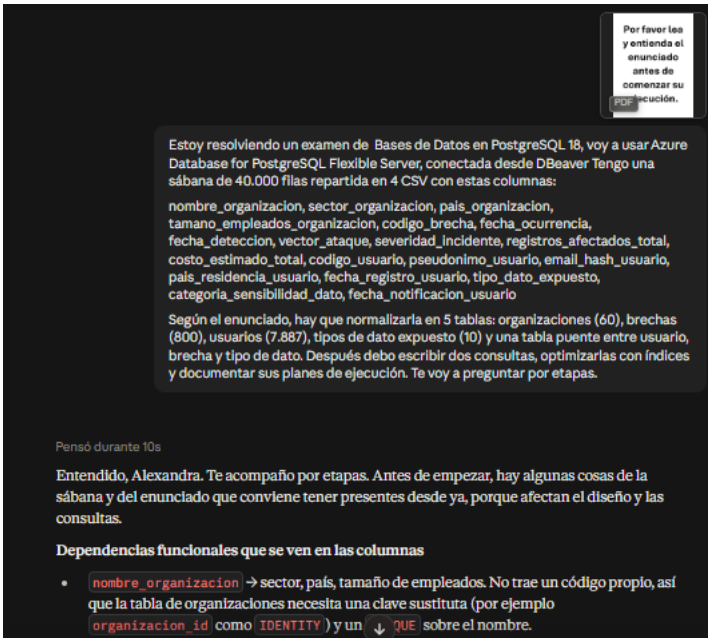
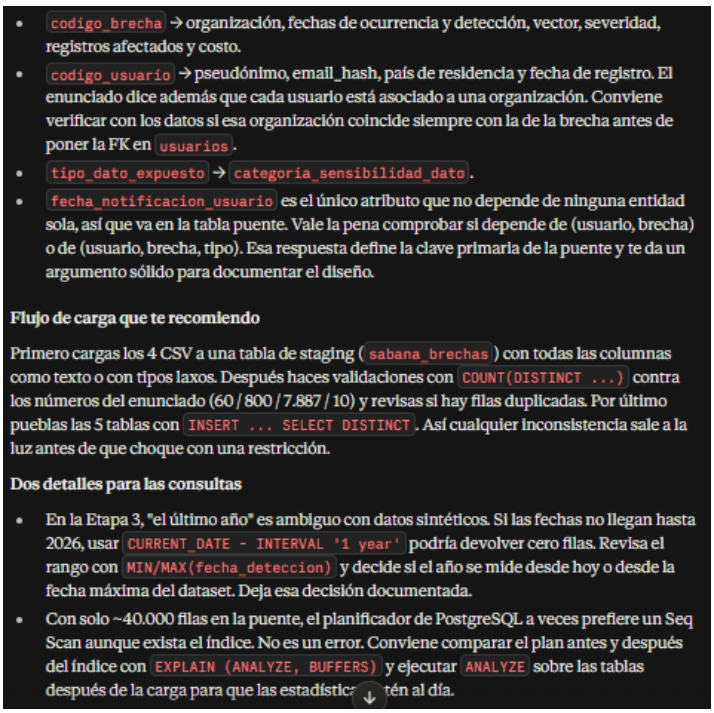
Examen 2 – Tópicos Avanzados de Base de Datos

Laura Londoño – 316613

13 de octubre de 2026

Interacción con la IA





Sobre Azure

Para cumplir la condición de "sin costos", usa el tier Burstable B1ms con el almacenamiento mínimo, que es lo que cubre el plan gratuito de 12 meses de una cuenta nueva o de Azure for Students. Detén el servidor cuando no lo estés usando.

Por último, recuerda que el enunciado exige capturas de pantalla de toda la interacción con la IA, no enlaces. Te sirve ir guardándolas desde ahora.

¿Por cuál etapa arrancamos?

hace / horas

arranquemos por la etapa 1, del abastecmiento, para el parcial pasado también tuve que hacer abstecimiento en postgres con dbeaver, te voy a adjuntar el archivo y me tienes que construir una guia que me permita terminar con la etapa 1

Ejecutó un comando

La respuesta de Claude fue interrumpida. Editar prompt Intentar nuevamente

Abastecimie... Londoño.docx

DOCX

Lo primero que te voy a pedir es que me ayudes con el abastecimiento desde AZure y Dbeaver, en el parcial pasado ya hecho

Guía Etapa 1 – Abastecimiento de PostgreSQL 18 en Azure y conexión con...
Claude Docs · Solo tú

Abrir

La Guía Etapa 1 está lista. Puedes editarla directamente o dejar comentarios y la ajusto.

Se me agotó la prueba gratuita de azure, que alternativas me recomiendas? Porque azure me está exigiendo la pocion de pay as you go y no estoy dispuesto a hacer eso, cierto que parece que no hay ninguna restricción en usar supabase en el documento

Se buscó en la web

Tienes razón en parte. El enunciado no prohíbe ningún proveedor: acepta instalación "en nube, con el proveedor de nube de su preferencia, siempre y cuando no genere costos". La restricción real está en el motor, porque el enunciado solo acepta PostgreSQL 18.x, y ahí Supabase probablemente no cumple. No encontré evidencia de que los proyectos alojados de Supabase ya ofrezcan PostgreSQL 18. Lo más reciente que publicaron es el paso de Postgres 15 a 17 como imagen por defecto en su versión self-hosted, en junio de 2026, y sus funciones nuevas se anuncian para Postgres 17 en adelante. Además, la comunidad sigue preguntando cuándo llegará el 18, que tenía una meta aproximada de enero de 2026. Si de todas formas quieres probarlo, crea un proyecto y ejecuta `SELECT version();` antes de invertirle tiempo. Pero no la usaría para el examen. supabase.com +2

Estas son las alternativas que te recomiendo, en orden:

Opción	Versión	Costo y tarjeta	Ventaja	Limitación
Azure for Students	PostgreSQL 18	USD 100 de crédito, sin tarjeta	La guía que ya tienes sirve casi igual	Solo una suscripción por estudiante
Neon	PostgreSQL 18 por defecto	Gratis permanente, sin tarjeta	Se crea en minutos	0,5 GB por proyecto
Docker local (`postgres:18`)	PostgreSQL 18	Gratis	Sin límites ni dependencias externas	Pierdes la bonificación por nube

1. Azure for Students. Es distinta de la prueba gratuita que se te agotó. Te da USD 100 en créditos sin tarjeta de crédito, válidos por un año y renovables cada año mientras seas estudiante activo. El único requisito es verificar tu condición académica con el correo institucional, que en tu caso sería el de @upb.edu.co. No te obliga a pasar a pago por uso: para seguir usando Azure después de agotar el crédito, tendrías que pasarte a Pay-As-You-Go contactando a soporte y dando una tarjeta, así que nunca te cobra sin que lo autorices. Un B1ms gasta una fracción pequeña de esos USD 100, sobre todo si lo detienes cuando no lo uses. La condición es que la oferta se limita a una suscripción por estudiante, así que solo te sirve si nunca la activaste. full api script +3

2. Neon. Es la mejor alternativa en nube si Azure for Students no te funciona. PostgreSQL 18 es la versión por defecto para todos los proyectos nuevos. El plan gratuito incluye 100 CU-horas de cómputo por proyecto al mes y 0,5 GB de almacenamiento por proyecto, y no pide tarjeta de crédito. Las 40.000 filas de la sábana caben con mucho margen. Además, el cómputo se apaga solo tras 5 minutos de inactividad, así que no tienes que acordarte de detener nada. Se conecta desde DBeaver con SSL igual que Azure. Lo único que debes tener en cuenta es que la primera consulta después de un rato inactivo tarda un par de segundos en despertar el servidor. Esa demora no afecta los planes de ejecución que vas a documentar. Multigres arrives as Neon s…

3. Docker local. Es el plan de respaldo. El enunciado lo acepta explícitamente, no depende de ninguna cuenta y te da control total. Lo dejaría como última opción solo porque pierdes la bonificación por infraestructura en nube.

Mi sugerencia es intentar primero Azure for Students con tu correo de la UPB. Si te dice que ya la usaste o no te valida, ve a Neon.

¿Quieres que adapte la guía a la opción que elijas?

hace / horas

voy a usar neon, generame la guia para esa interfaz, asegurandonos que cumpla con todo o necesario para terminar co la fase 1

Make doc changes (batch)

Estructura de la guía de Neon lista; ahora reviso cómo cubre el enunciado.

Se leyeron 5 páginas, se buscó en la web

Guía Etapa 1 – Abastecimiento de PostgreSQL 18 en Neon y conexión con…
Claude Docs · Solo tú

Abrir

Ahora lleno qué pide el enunciado y cómo lo cubre Neon.

Usó la integración de Claude Docs

La Guía Etapa 1 con Neon está lista. Puedes editarla directamente o comentar lo que quieras ajustar.

la parte de crear la conexión me está dando problemas, no se que parte copiar en dbeaver y además no me daj configurar el modo verify full(aunque no parece necesitarse para esta entrega), mira el snippet que me da Neon: postgresql://tadb_examen02_owner:contraseña@ep-divine-snow-b41layia.c-6.us-east-2.aws.neon.tech/tadb_examen02?sslmode=require&channel_binding=require

Usó la integración de Claude Docs, herramientas cargadas

Esa cadena trae todos los datos que pide DBeaver; solo hay que separarlos en campos, porque DBeaver no acepta la cadena completa en el campo URL. En la pestaña **Principal**, con **Conectar por: Host**, queda así:

Campo en DBeaver	Valor (sale de tu cadena)
Host	`ep-divine-snow-b41layia.c-6.us-east-2.aws.neon.tech` (lo que va entre `@` y `/`)
Puerto	`5432` (no aparece en la cadena; es el de siempre)
Base de datos	`tadb_examen02` (entre `/` y `?`)
Usuario	`tadb_examen02_owner` (entre `postgresql://` y `:`)
Contraseña	lo que va entre `:` y `@`

Fíjate que tu usuario es `tadb_examen02_owner` y no `neondb_owner`. Neon le pone al rol el nombre de la base, y como la nombraste al crear el proyecto, el rol cambió. Ya lo corregí en la guía. Tu host además es el directo (no tiene `-pooler`), que es el que necesitas.

Sobre el SSL, tienes razón: el enunciado no exige `verify-full`. Usa **require**, que es lo mismo que trae tu cadena con `sslmode=require`, y la conexión queda cifrada igual, porque Neon rechaza cualquier conexión sin TLS. La prueba del Paso 6.2 (conectar sin SSL y ver que falla) es la evidencia de eso. El `channel_binding=require` no lo configures, DBeaver no lo necesita.

Si de todas formas quieres intentar `verify-full`, lo más probable es que el selector estuviera bloqueado porque primero hay que marcar **Usar SSL**; los demás campos de la pestaña se activan solo después. Si aun así no te deja, ve a **Propiedades del driver** y agrega `sslmode` = `verify-full` y `sslrootcert` = `C:\certs\neon\isrgrootx1.pem`. Dejé ambas opciones en el paso 5.2 de la guía, junto con una tabla de cómo leer la cadena de conexión.

Bien hecho al reemplazar la contraseña por "contraseña" en el mensaje. Mantenla igual de fuera de las capturas del PDF y del repositorio de GitHub.

Abasteclmie... e datos.docx

DOCX



PDF

quiero que documentes este documento de esta entrega como cmente este otro documento de una entrega pasada, menciona lo de la seguridad innata de Neon, muy importante que escribas como escribo yo

Se ejecutaron 10 comandos, leyó 7 archivos, memoria leída



Te dejé documentado el Word con tus mismas capturas, siguiendo la estructura de la entrega pasada. Hay dos secciones con pasos numerados, un título en negrita con dos puntos y una explicación corta. El texto va en primera persona del plural, con frases sencillas como las del documento de Supabase.

La seguridad de Neon quedó como el paso 4 de la primera sección. Explica que viene activada por defecto y no se puede apagar, con cuatro capas: TLS obligatorio, cifrado AES-256 en reposo, contraseñas de 60 bits de entropía y el proxy que filtra accesos. También explica por qué no usaste la restricción por IP: solo está en el plan pago. En la parte de DBeaver aclaré que no tuviste que configurar el SSL a mano, porque Neon ya obliga el cifrado. Eso coincide con tus capturas, donde no aparece la pestaña SSL.

Revisa tres cosas antes de entregar:

- **ID de SIGAA:** agregué una línea con tu nombre al inicio, pero el ID quedó resaltado en amarillo como "[completar]". El enunciado exige que todos los entregables estén marcados.
- **Captura de la cadena de conexión:** se tomó con Connection pooling activado, por eso el host termina en `-pooler`, mientras que en DBeaver usaste el host directo. El texto lo explica, pero si puedes, repite esa captura con el pooling apagado para que todo coincida.
- **Un error del documento pasado que no copié:** allí se dice que `verify-full` valida "que todos los clientes tengan el certificado". En realidad es al revés: el cliente valida el certificado del servidor. No lo repetí, para que no te lo marquen.

Abastecimiento base de datos
Documento · DOCX
Descargar

hace 5 horas

perfecto, ya podemos seguir con la etapa 2:
El enunciado pide:
1. Un diagrama relacional en PNG con fondo blanco: nombres de tablas y campos, PK y FK.
2. Un script .sql (UTF-8) con: creación de usuario y esquema, privilegios mínimos, tablas, vistas, secuencias, índices, restricciones, funciones y procedimientos.
Los dos van marcados con: Laura Londoño – ID SIGAA [ID SIGAA].

Modelo (5 tablas, ya verificado contra los datos):
- organizaciones(id, nombre, sector, pais, tamano_empleados)

Mostrar más

Se ejecutaron 7 comandos, creó un archivo, editó un archivo

Script listo. Lo probé completo en un PostgreSQL local con datos sintéticos: la carga funciona, se puede ejecutar dos veces sin duplicar, y el usuario lector puede consultar las tablas, la vista y la función, pero no puede leer la staging, ejecutar la carga, borrar datos ni crear tablas. Esa prueba me sirvió para corregir un error real: el procedimiento no encontraba las tablas desde otra sesión, y ya quedó resuelto con `SET search_path`.

Validación contra el enunciado

El diseño cumple la Etapa 2. El script incluye todo lo que pide el enunciado: usuario y esquema, privilegios mínimos, tablas, vistas, secuencias, índices, restricciones, funciones y procedimientos. Las 5 tablas también alcanzan para las etapas siguientes. La Etapa 3 usa la fecha de detección, la severidad, la categoría de sensibilidad, el nombre de la organización y el conteo de usuarios distintos desde `exposiciones`. La Etapa 4 usa el código de usuario, la brecha, el tipo de dato, la categoría, la organización y la `fecha_notificacion`.

Me aparté de tus indicaciones en dos puntos:

- **Índice de `exposiciones.id_usuario`:** no lo creé. `id_usuario` es la primera columna de la PK, así que el índice de la PK ya lo cubre. Otro índice sería redundante, y eso puede pesar en las etapas de desempeño. Lo dejé explicado en un comentario dentro del script.
- `tamano_empleados`: quedó como `VARCHAR(50)`. No sé si los datos traen un número o un rango tipo "51-200", y el texto acepta los dos casos.

Antes de ejecutar la carga, haz una comprobación en la staging. Los CHECK asumen los valores `'Baja'`, `'Media'`, `'Alta'` y `'Critica'` tanto en severidad como en categoría, y las fechas se convierten asumiendo formato `AAAA-MM-DD`. Confírmalo con esto:

```sql
SELECT DISTINCT severidad_incidente FROM brechas_seguridad.staging_sabana;
SELECT DISTINCT categoria_sensibilidad_dato FROM brechas_seguridad.staging_sabana;
SELECT fecha_deteccion, fecha_notificacion_usuario FROM brechas_seguridad.staging_saba
```

Si algún valor es distinto, ajusta la lista del CHECK correspondiente.

Cómo ejecutar el script en DBeaver

Ábrelo sobre la conexión a `tadb_examen02` y ejecútalo con **Ejecutar script** (Alt+X), no con Ctrl+Enter, porque Ctrl+Enter solo ejecuta una sentencia. Antes, cambia la contraseña del rol `lector_brechas` en la sección 9. Si necesitas volver a empezar, ejecuta `DROP SCHEMA brechas_seguridad CASCADE;` y `DROP OWNED BY lector_brechas; DROP ROLE lector_brechas;`. Después de eso tendrás que reimportar los CSV.

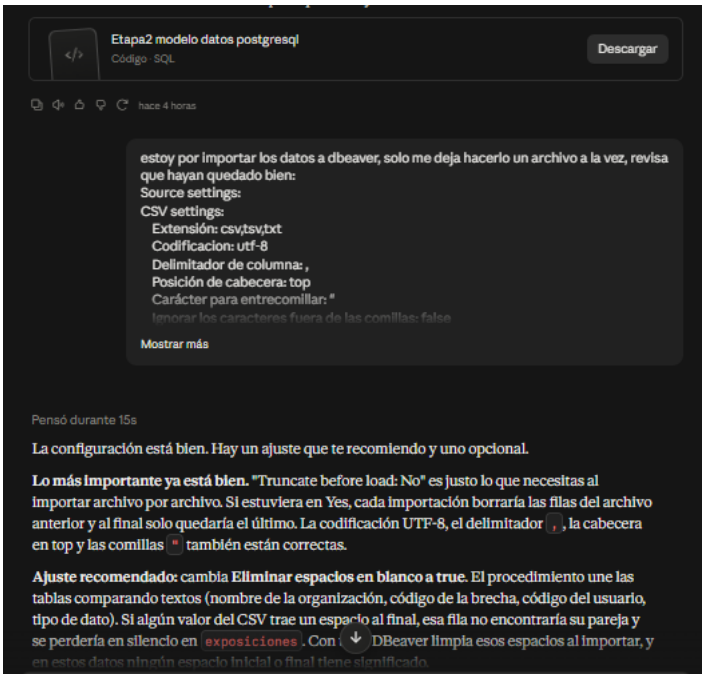
Importar los 4 CSV a la staging

1. Descomprime los 4 archivos `.zip`.
2. En el navegador de DBeaver, actualiza (F5) el esquema `brechas_seguridad`. Luego haz clic derecho en `staging_sabana` y elige **Importar datos**, con origen **CSV**.
3. Selecciona los 4 CSV a la vez. DBeaver crea una línea de mapeo por archivo, y todas deben apuntar a `staging_sabana`.
4. En la configuración de lectura, usa codificación **UTF-8**, delimitador `,`, encabezado **arriba** y comillas `"`.
5. En el mapeo, las columnas deben quedar como **existing**. Se asocian solas porque se llaman igual que los encabezados del CSV.
6. Ejecuta la importación y verifica con `SELECT count(*) FROM brechas_seguridad.staging_sabana;`. Deben salir 40.000 filas.
7. Ejecuta `CALL brechas_seguridad.sp_cargar_modelo();` y luego `SELECT * FROM brechas_seguridad.fn_conteo_tablas();`. Deben salir 60, 800, 7.887 y 10.

Exportar el diagrama

1. Pasa DBeaver al tema claro, para que el fondo quede blanco: **Ventana → Preferencias → Interfaz de usuario → Apariencia → Tema: Light**.
2. Haz clic derecho en el esquema `brechas_seguridad` y elige **Ver diagrama**.
3. Selecciona `staging_sabana` y bórrala del diagrama con Supr. Solo desaparece de la vista, no de la base, y el diagrama debe mostrar únicamente las 5 tablas.
4. Haz clic derecho sobre el fondo y elige **Crear nota**. Escribe ahí la marca con el nombre y el ID de SIGAA.
5. Usa el botón **Exportar diagrama** de la barra del diagrama y guárdalo como **PNG**.

Por último, en el script usé la marca que pediste, "Laura Londoño - ID SIGAA [ID SIGAA]", pero el documento de la Etapa 1 quedó a nombre de María Alexandra Vasco. Si están trabajando en pareja, el enunciado pide que en los entregables figuren los dos, así que conviene marcar ambos nombres e IDs en todos los archivos para que no [illegible]ra inconsistencias.



Opcional: pon **Use multi-row Insert en Yes**. Neon está en Ohio y cada viaje de red cuenta, así que agrupar 500 filas por INSERT acelera bastante la carga. Con No también funciona, solo que más lento.

El resto (formato de fecha, longitud de columna, auto-generated columns) no afecta, porque la tabla ya existe y todas sus columnas son TEXT.

En la página de mapeo, antes de ejecutar, confirma que el destino sea `staging_sabana` y que las 19 columnas digan **existing**. Si alguna dice **skip** o **new**, el encabezado de ese CSV no coincidió con el nombre de la columna.

Después de cada archivo, ejecuta:

```sql
SELECT count(*) FROM brechas_seguridad.staging_sabana;
```

El conteo debe ir subiendo con cada lote hasta llegar a 40.000 al final. Si por error importas un archivo dos veces, ejecuta `TRUNCATE brechas_seguridad.staging_sabana;` y vuelve a empezar.

Cuando termines los 4 archivos, haz esta revisión antes del `CALL`. DBeaver detecta los tipos de las columnas con las 100 filas de muestra, y a veces convierte números grandes a notación científica (`1.0E7`), que luego no se puede convertir a BIGINT:

```sql
SELECT count(*) FILTER (WHERE registros_afectados_total !~ '^\d+$')      AS registros_
       count(*) FILTER (WHERE costo_estimado_total !~ '^\d+(\.\d+)?$')  AS costos_raro
       count(*) FILTER (WHERE fecha_deteccion !~ '^\d{4}-\d{2}-\d{2}')  AS fechas_rara
FROM brechas_seguridad.staging_sabana;
```

Si las tres columnas dan 0, puedes ejecutar `CALL brechas_seguridad.sp_cargar_modelo();` sin problema. Si alguna da más de 0, pásame un par de esos valores y ajustamos la conversión en el procedimiento.

esto es lo que corro justo después de cargar los datos a la tabla de staging cierto?

```
-- ------------------------------------------------------------
-- 6. Procedimiento: carga de la staging a las 5 tablas
-- ------------------------------------------------------------
-- Se puede ejecutar más de una vez: ON CONFLICT evita duplicados.
-- SET search_path hace que encuentre las tablas desde cualquier sesión.
CREATE PROCEDURE sp_cargar_modelo()
LANGUAGE plpgsql
SET search_path = brechas_seguridad
AS $$
```

Mostrar más

Pensó durante 1s

No, ese bloque no se vuelve a correr. Ese bloque crea el procedimiento, y eso ya pasó cuando ejecutaste el script completo; por eso la tabla `staging_sabana` ya existía para importar los CSV. Si lo ejecutas otra vez, PostgreSQL responde con un error de que el procedimiento ya existe.

Después de cargar los 4 archivos solo tienes que **llamarlo**. El orden es este:

sql

```sql
-- 1. Confirmar que están las 40.000 filas
SELECT count(*) FROM brechas_seguridad.staging_sabana;

-- 2. Revisión de formatos (las tres columnas deben dar 0)
SELECT count(*) FILTER (WHERE registros_afectados_total !~ '^\d+$')      AS registros_
       count(*) FILTER (WHERE costo_estimado_total !~ '^\d+(\.\d+)?$')  AS costos_raro
       count(*) FILTER (WHERE fecha_deteccion !~ '^\d{4}-\d{2}-\d{2}')  AS fechas_rara
FROM brechas_seguridad.staging_sabana;

-- 3. Ejecutar la carga a las 5 tablas
CALL brechas_seguridad.sp_cargar_modelo();

-- 4. Verificar el resultado
SELECT * FROM brechas_seguridad.fn_conteo_tablas();
```

En el paso 4 deberías ver 60 organizaciones, 800 brechas, 7.887 usuarios y 10 tipos de dato. El número de `exposiciones` no lo da el enunciado: será 40.000 si cada fila de la sábana es una combinación distinta de usuario, brecha y tipo de dato, o un poco menos si había filas repetidas. Si el `CALL` da error, pégame el mensaje completo y lo revisamos.

SQL Error [42703]: ERROR: column "registros_afectados_total" does not exist
Position: 91

Pensó durante 9s

Bien que salió en la revisión y no en el `CALL`: el procedimiento usa esa misma columna, así que habría fallado igual, pero a mitad de la carga.

El error dice que en `staging_sabana` no hay ninguna columna que se llame exactamente `registros_afectados_total`. Hay tres causas probables:

1. **El encabezado real del CSV tiene otro nombre.** Por ejemplo `registros_afectados` o `registros_afectados_totales`. En ese caso la columna puede haber quedado en **skip** en el mapeo y sus datos no se importaron.
2. **DBeaver creó una tabla o columna nueva** al importar (mapeo **new** en vez de **existing**), y los datos quedaron en otro lugar.
3. **El nombre quedó con mayúsculas o espacios.** Si DBeaver creó la columna como `"Registros_afectados_total"`, PostgreSQL la trata como un nombre distinto.

Para saber cuál es, ejecuta esta consulta. Busca en toda la base cualquier columna que contenga "registros":

```sql
SELECT table_schema, table_name, column_name
FROM information_schema.columns
WHERE column_name ILIKE '%registros%';
```

Ejecuta también esta, para ver cómo quedó la staging completa:

```sql
SELECT column_name
FROM information_schema.columns
WHERE table_schema = 'brechas_seguridad'
  AND table_name = 'staging_sabana'
ORDER BY ordinal_position;
```

Según lo que salga, la solución es una de estas:

- **La columna está en `staging_sabana` con otro nombre y tiene datos:** basta con renombrarla, sin tocar el procedimiento ni reimportar:

```sql
ALTER TABLE brechas_seguridad.staging_sabana
    RENAME COLUMN <nombre_actual> TO registros_afectados_total;
```

- **La columna está en `staging_sabana` pero vacía, o los datos quedaron en otra tabla:** hay que vaciar la staging con `TRUNCATE brechas_seguridad.staging_sabana;` y reimportar los 4 archivos. En el mapeo, asigna a mano esa columna del CSV a `registros_afectados_total`.

Pásame el resultado de las dos consultas y te digo exactamente cuál aplicar. Si otra columna también salió con un nombre distinto, la corregimos de una vez.

hace 3 horas

SELECT table_schema, table_name, column_name
FROM information_schema.columns
WHERE column_name ILIKE '%registros%';
brechas brechas_seguridad registros_afectados
staging_sabana brechas_seguridad registros_afectados_total
SELECT column_name
FROM information_schema.columns
WHERE table_schema = 'brechas_seguridad'
AND table_name = 'staging_sabana'
ORDER BY ordinal_position

Mostrar más

Pensó durante 37s

Entonces la tabla está bien. El problema está en cómo se ejecutó la consulta de revisión, no en los datos.

El "Position: 91" lo confirma. Si se cuenta desde la línea de comentario `-- 2. Revisión de formatos...` (con los saltos de línea de Windows), el carácter 91 es justo donde empieza `registros_afectados_total` dentro de la consulta de revisión. O sea, PostgreSQL recibió esa consulta, pero en ese momento no encontró la columna en la tabla que estaba consultando. Si la columna existe, lo más probable es que esa pestaña del editor estuviera apuntando a otra conexión o a otra base de datos distinta a la de las consultas de diagnóstico.

Revisa tres cosas, en este orden:

1. **La conexión de la pestaña.** En la barra superior del editor SQL, DBeaver muestra la conexión y la base activas. Deben decir `Neon PG18 - Examen 02` y `tadb_examen02`. Si dice otra conexión, o `neondb`, cámbiala ahí mismo.
2. **Una lectura directa de la columna,** en la misma pestaña donde salió el error:

```sql
SELECT registros_afectados_total
FROM brechas_seguridad.staging_sabana
LIMIT 3;
```

3. **Que el nombre no tenga caracteres invisibles.** El resultado debe ser 25:

```sql
SELECT length(column_name)
FROM information_schema.columns
WHERE table_schema = 'brechas_seguridad'
  AND table_name = 'staging_sabana'
  AND column_name ILIKE '%registros%'
```

Si la lectura directa funciona y la longitud es 25, puedes pasar de una vez al `CALL brechas_seguridad.sp_cargar_modelo();`. La revisión de formatos era solo una precaución. Te corrijo algo que dije antes: si el `CALL` falla, no deja la carga a medias. El procedimiento no hace COMMIT por dentro, así que un error deshace todo lo que alcanzó a insertar y las 5 tablas quedan vacías como estaban. Ejecutarlo directamente es seguro, y si algún valor no se puede convertir, el mensaje de error te dirá cuál.

Si la lectura directa también da el mismo error, pásame una captura de la pestaña completa del editor, con la barra de conexión visible, y lo revisamos.

después de tener bien cargados los datos debería eliminar la tabla de staging verdad, ya no sirve para nada en el resto del documento

Pensó durante 6s

No es necesario, y te recomiendo no eliminarla, por lo menos hasta terminar el examen.

No estorba. Las consultas de las Etapas 3 y 4 no la tocan, así que no afecta los planes de ejecución ni los costos que vas a documentar. Tampoco expone nada, porque el usuario `lector_brechas` no tiene permisos sobre ella. Y el espacio no preocupa: 40.000 filas de texto ocupan unos pocos MB del GB del plan gratuito.

Te puede ahorrar trabajo. Si en las etapas siguientes necesitas ajustar algo del modelo (un tipo de dato, un CHECK), puedes vaciar las 5 tablas y volver a ejecutar el `CALL` sin reimportar los 4 CSV. Si la borras, ese escenario te obliga a repetir la importación archivo por archivo.

Debe seguir en el script. El procedimiento `sp_cargar_modelo` lee de la staging, así que el `.sql` que entregas necesita el `CREATE TABLE staging_sabana` para que el modelo se pueda reconstruir desde cero. El profesor, al ejecutar tu script, también necesita esa tabla para cargar los datos. Si en la base borraras la tabla pero no el procedimiento, el procedimiento quedaría

apuntando a una tabla que no existe.

Si al final del examen quieres limpiar, usa `TRUNCATE` en lugar de `DROP`:

```sql
TRUNCATE brechas_seguridad.staging_sabana;
```

Así se liberan los datos, pero la estructura queda igual a lo que dice el script y el procedimiento sigue siendo válido.

ETAPAS 3 Y 4 – Consultas (PostgreSQL 18 en Azure, DBeaver)
El enunciado pide:
1. Un script .sql con las dos consultas, incluyendo los elementos de mejora de desempeño usados.
2. Un Word (.docx) con, por cada consulta: captura del plan de ejecución donde se vea claramente el árbol de operaciones, las filas y el costo, y captura de los registros resultantes.
Los índices adicionales se documentan en el script del modelo, como indica el enunciado.
Todo marcado con: Laura Londoño – ID SIGAA [ID SIGAA].

Mostrar más

Se ejecutaron 5 comandos, creó un archivo, archivos c… artidos

Te dejé dos archivos: el script nuevo de consultas y el del modelo, que ahora termina con una sección 11 de índices. Probé todo en un PostgreSQL local con 40.000 filas sintéticas. Las dos consultas corren bien, y los planes usan los índices como se esperaba.

Validación contra el enunciado

La Consulta 1 cumple los cuatro puntos que pide la Etapa 3: filtra por rango de fechas en `brechas`, filtra por severidad y por categoría de sensibilidad, hace join con la tabla puente para contar usuarios distintos y hace join con organizaciones para traer el nombre. Tu decisión de contar todos los usuarios de la brecha, con el dato crítico como condición en el EXISTS, es la lectura correcta de "número total de usuarios distintos afectados por esa brecha". Agregué el código de la brecha como segundo criterio de orden, para que los empates salgan siempre en el mismo orden.

La Consulta 2 trae los cinco datos de la Etapa 4. Un detalle que dejé explícito en un comentario: la organización responsable es la de la **brecha**, no la organización a la que está asociado el usuario. Por eso el join con organizaciones va por `b.id_organizacion`.

Índices de la sección 11

- `(severidad, fecha_deteccion)` **en brechas:** responde al filtro de fechas que pide el enunciado. Con solo 800 brechas, el planificador puede preferir no usarlo y filtrar las filas directamente, como pasó en mi prueba. No es un error: con tablas tan pequeñas, leerlas completas es más barato que pasar por un índice. Si te pasa, explícalo así en el Word.
- `(id_brecha, id_usuario)` **en exposiciones:** este sí se notó en la prueba. El conteo de usuarios distintos salió como *Index Only Scan*, o sea, leyendo solo el índice sin tocar la tabla. Reemplaza al índice simple de `id_brecha`, porque cubre también la FK.
- **Consulta 2:** no necesitó índices nuevos. Usa el UNIQUE de `usuarios.codigo` y la PK de `exposiciones`, y en la prueba tardó 0,13 ms.

Como tu base ya tiene todo lo demás, ejecuta solo la sección 11: selecciónala y pulsa Alt+X. Los nombres llevan el esquema, así que funciona sola. Termina con `ANALYZE`, que actualiza las estadísticas para que el planificador conozca los …os cargados.

Revisa esto antes de tomar capturas

```sql
SELECT MIN(fecha_deteccion), MAX(fecha_deteccion)
FROM brechas_seguridad.brechas;
```

Si la fecha máxima es anterior al 13 de octubre de 2025, la Consulta 1 devolverá cero filas con `CURRENT_DATE`. Avísame y la ajustamos para medir el año desde la última fecha del dataset. Además, como `CURRENT_DATE` cambia cada día, toma la captura del plan y la de los resultados el mismo día, para que coincidan.

En tu mensaje dice "PostgreSQL 18 en Azure", pero los scripts dicen Neon, que es lo que estás usando. Revisa que el Word también diga Neon.

Cómo ver el plan en DBeaver

Para la captura del árbol:

1. Selecciona el texto de la consulta 1.a, desde el `SELECT` hasta el punto y coma.
2. Pulsa **Ctrl+Shift+E** (Explicar plan de ejecución). Si aparece una ventana de opciones, marca **ANALYZE**.
3. Se abre la pestaña **Plan de ejecución** con el árbol y las columnas de tipo de nodo, entidad, costo, filas, tiempo y condición. Expande todos los nodos y haz la captura, con las columnas de costo y filas visibles. Ese es el árbol de operaciones que pide el enunciado.

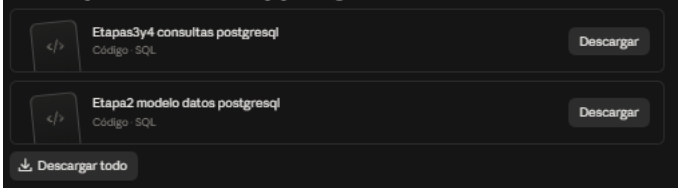
Para la versión en texto, que conviene como captura complementaria:

1. Ejecuta el bloque 1.b con **Ctrl+Enter**.
2. El resultado sale en una sola columna, `QUERY PLAN`. Cambia a la vista **Texto** con el botón de la barra izquierda del panel de resultados, para ver el plan completo con costos y filas reales.

Para los registros resultantes:

1. Ejecuta el bloque 1.a con **Ctrl+Enter**.
2. Captura la grilla, incluyendo la barra inferior donde dice cuántas filas se obtuvieron.

Repite lo mismo con la Consulta 2. Si el árbol queda muy ancho para la página del Word, el enunciado permite usar un tamaño de papel más grande.



hace 1 hora

por que me volviste pasar el script de la etapa 2, ese ya n estaba completado?

Pensó durante 1s

Te lo volví a pasar porque cambió. En tu mensaje anterior pediste que los índices de las consultas se agregaran **al final del script del modelo**, y el enunciado dice lo mismo: los índices se documentan en el script de creación del modelo. Por eso el archivo ahora tiene una sección 11 nueva con los índices de las Etapas 3 y 4 y los A ZE. Las secciones 1 a 10 quedaron

nueva con los índices de las Etapas 3 y 4 y los `ANALYZE`. Las secciones 1 a 10 quedaron exactamente igual.

En la práctica son dos cosas distintas:

- **En la base de datos:** ejecuta solo la sección 11. No vuelvas a correr el script completo, porque el esquema, las tablas y el rol ya existen y daría error desde la primera línea.
- **En el repositorio:** reemplaza tu archivo `etapa2_modelo_datos_postgresql.sql` por esta versión. Esa es la que se entrega, porque el profesor debe ver los índices de desempeño documentados ahí.

Revisa esto antes de tomar capturas sql SELECT MIN(fecha_deteccion),...

mira el resultado de la ejecución de ese comando SQL:
"min","max"
2021-10-06,2026-09-22

hace 1 hora

Pensó durante 3s

Con ese resultado no hay que cambiar nada. La consulta sí va a devolver filas.

Con `CURRENT_DATE`, la ventana del "último año" va desde el 13 de octubre de 2025 hasta hoy. La brecha más reciente se detectó el 22 de septiembre de 2026, así que caen dentro casi once meses de datos. Además, los datos llegan hasta un día después de la fecha del enunciado (el archivo es del 21 de septiembre de 2026), lo que sugiere qu[illegible] generaron pensando en que el último año se mida desde la fecha actual. Tu definición encaja con eso.

Lo único a tener en cuenta es que, como `CURRENT_DATE` avanza cada día, la ventana también se va corriendo. Las brechas detectadas justo al inicio de la ventana irán saliendo del resultado con el paso de los días. Por eso es importante tomar la captura del plan y la de los registros el mismo día, como te dije antes. Si quieres que el Word lo deje claro, puedes escribir la fecha en que ejecutaste la consulta junto a las capturas.



Cómo ver el plan en DBeaver Para la captura del árbol: Selecciona el texto de la...

esto o debo hacer para cada conusulta, mira el árbol que me dió para la cnsulta 1.a. así está bien, eso no me parece un árbol

Pensó durante 9s

Sí es un árbol, solo que DBeaver lo dibuja como una lista con sangría, no con cajas y flechas. Cada nivel de sangría es un hijo del nodo de arriba, y las flechitas `v` sirven para expandir o colapsar ramas. Se lee de adentro hacia afuera: los nodos más indentados se ejecutan primero y le pasan sus filas al nodo padre, hasta llegar al `Sort` de la raíz, que es el resultado final.

Y sí, tienes que hacerlo para las dos consultas: una captura del plan de la 1.a y otra de la 2.a, cada una con su captura de registros resultantes.

El problema de tu captura es que la columna **Tipo de Nodo** está muy angosta y corta los nombres ("Nested Loo", "Inde", "Ir", "H", "Memoiz"). El enunciado pide que el árbol se vea **claramente**, así que arréglala antes de capturar:

1. Haz doble clic en la pestaña **Plan de ejecu[illegible]** -1 para maximizar el panel

2. Arrastra el borde derecho de la columna **Tipo de Nodo** hasta que se lean los nombres completos, como `Index Scan using idx_brechas_severidad_fecha_deteccion` o `Index Only Scan using idx_exposiciones_id_brecha_id_usuario`.
3. Ensancha también **Condición**, para que se vean los filtros completos.
4. Si no cabe todo en una sola captura, toma dos: una con el árbol y otra con las columnas de la derecha.

En la barra vertical de la izquierda, debajo de **Simple**, hay otro ícono. En las versiones recientes de DBeaver, ese ícono abre una vista gráfica del plan con cajas y conexiones, que se parece más a un árbol tradicional. Si tu versión la tiene, puedes agregar esa captura, pero la lista con sangría ya cumple.

Además, el plan salió muy bien para documentar:

- **Los índices nuevos sí se usan.** Las filas amarillas son accesos por índice. El de `brechas`, con la condición sobre `severidad`, es `idx_brechas_severidad_fecha_deteccion`. El `Index Only` de `exposiciones` es `idx_exposiciones_id_brecha_id_usuario`, que resuelve el conteo de usuarios leyendo solo el índice.
- **El resultado:** 101 brechas, con un costo total de 1244–1257 y unos 5 ms de ejecución.

Para tener los nombres completos de cada nodo en texto, ejecuta también el bloque 1.b con Ctrl+Enter y cambia el resultado a la vista **Texto**. Esa captura complementa muy bien la del árbol.

hace 1 hora

no entendí bien por que las consultas están separadas en 1.a,1.b,2.a y 2.b, por qué pasa esto, cuales debo correr normal y cuales con ctl shift e?

Cada consulta aparece dos veces porque son la misma consulta usada para dos cosas distintas:

- **.a es la consulta tal cual.** Al ejecutarla devuelve los **registros** (las brechas, los usuarios, etc.).
- **.b es la misma consulta con `EXPLAIN (ANALYZE)` adelante.** Esa palabra le dice a PostgreSQL que no muestre los registros sino **cómo los obtuvo**: el plan de ejecución con costos, filas y tiempos, en texto.

Las dos van en el script porque el enunciado pide las consultas junto con los elementos de mejora de desempeño, y el `EXPLAIN (ANALYZE)` es la forma de evidenciar ese desempeño dentro del `.sql`.

Para cada consulta, se ejecutan así:

Bloque	Cómo ejecutarlo	Qué obtienes	Captura para el Word
1.a / 2.a	**Ctrl+Enter**	La grilla con los registros	Registros resultantes
1.a / 2.a	Seleccionar + **Ctrl+Shift+E**	El árbol en la pestaña Plan de ejecución	Árbol con costo y filas
1.b / 2.b	**Ctrl+Enter**	El plan en texto (columna QUERY PLAN)	Complementaria, opcional

Con **Ctrl+Shift+E**, DBeaver le agrega el `EXPLAIN` por su cuenta y te muestra el resultado como árbol. Por eso ese atajo se usa sobre la consulta **.a**, que no lo tiene escrito. Nunca lo uses sobre un bloque **.b**: quedaría un `EXPLAIN` dentro de otro y PostgreSQL daría error.

El orden para cada consulta queda así:

1. Selecciona el bloque .a y pulsa Ctrl+Enter. Captura los registros.
2. Con el mismo bloque .a seleccionado, puls l+Shift+E. Captura el árbol.
3. Si quieres la versión en texto, selecciona el bloque .b, pulsa Ctrl+Enter y cambia a la vista Texto. Haz esa captura.

Las capturas 1 y 2 son las obligatorias. La 3 es un respaldo útil porque muestra los nombres completos de los índices sin tener que ensanchar columnas.

hace 59 minutos

Necesito ayuda con el archivo gitignore del repositorio, que debería ir allí? si no tengo ningún certificado, o llave?

Creó un archivo, archivos compartidos

El `.gitignore` sí hace falta aunque no tengas certificados ni llaves. El enunciado pide uno "correspondiente al tipo de contenido que se almacenará", así que debe pensarse para lo que tiene tu repositorio: scripts `.sql`, documentos `.docx` y `.pdf`, el diagrama `.png` y el `README.md`. El archivo que te armé excluye cinco grupos:

- **Credenciales y certificados:** es preventivo. Hoy no tienes ninguno en la carpeta, pero si mañana queda ahí el `isrgrootx1.pem` o un `.env` con la cadena de conexión de Neon, Git no lo sube.
- **Datos de origen:** los CSV y los ZIP de la sábana. Los entrega el profesor, pesan, y la base se puede reconstruir con tus scripts.
- **Archivos de DBeaver:** su configuración de conexiones puede guardar contraseñas.
- **Temporales de Office:** los `~$...docx` que Word crea mientras tienes un documento abierto.
- **Archivos del sistema y de editores:** `Thumbs.db`, `desktop.ini`, `.vscode/`, etc.

Ten en cuenta tres cosas:

1. **Contraseña en el script del modelo.** El `.gitignore` no protege lo que está *dentro* de los archivos que sí subes. Si en la sección 9 de `etapa2_modelo_datos_postgresql.sql` reemplazaste `'Cambiar_Por_Una_Clave_Larga_2026'` por la contraseña real de `lector_brechas`, vuelve a dejar el texto de ejemplo antes de hacer commit. Revisa también que la contraseña de `tadb_examen02_owner` no aparezca en ningún archivo ni captura.

2. **El nombre del archivo.** Al descargarlo, Windows o el navegador pueden quitarle el punto inicial y dejarlo como `gitignore`. Debe llamarse exactamente `.gitignore` y estar en la raíz del repositorio. Si el Explorador de Windows no te deja renombrarlo, ejecuta esto en `cmd` dentro de la carpeta:

```
ren gitignore .gitignore
```

3. **Archivos que ya subiste.** El `.gitignore` solo afecta a los archivos nuevos. Si ya habías hecho commit de algún CSV, ZIP o temporal de Word, Git lo seguirá rastreando. Sácalo del repositorio, sin borrarlo de tu computador, con:

```
git rm --cached nombre_del_archivo
git commit -m "Quitar archivos excluidos por .gitignore"
```

.gitignore
GITIGNORE

Descargar

tadb202620_examen_02

Carpeta

quiero que me generes el archivo README para la entrega, solo falta el archivo .gitignore qu eya lo tengo pero lo subiré luego

hace 2 minutos

Ejecutado 4 comandos (2 errores), leyó 3 archivos, creado un archivo +80 -0 >

Creé el README.md. Para escribirlo leí los dos scripts, el diagrama y los dos documentos. Los datos del README salen de ahí, incluido que la infraestructura final quedó en **Neon** con PostgreSQL 18.6, no en Azure. Las consultas devuelven 101 y 20 filas, que coinciden con lo que yo había calculado.

En el README quedó el marcador `[ID SIGAA]`; reemplázalo con tu ID.

Revisando la carpeta encontré varias cosas que pueden afectar la nota:

1. **Falta el PDF de la interacción con IA.** Sin él la nota e

Revisando la carpeta encontré varias cosas que pueden afectar la nota:

1. **Falta el PDF de la interacción con IA.** Sin él la nota es 0.
   - En el README lo dejé como `Laura_Londoño_Interaccion_IA.pdf`.
   - Guárdalo con ese nombre o cambia el nombre en el README.
   - Debe tener capturas de toda la conversación, incluida esta.
2. **No aparece tu ID de SIGAA en ningún entregable.**
   - Los dos `.sql` no tienen encabezado con tu nombre.
   - El diagrama no tiene ni nombre ni ID. En DBeaver puedes agregar una nota al diagrama y volver a exportarlo.
3. **La carpeta no es un repositorio propio.** Git detecta uno en `C:\Users\User` (tu carpeta de usuario), sin commits. Si haces `git add` desde la carpeta del examen, los archivos se agregan a ese repositorio. Antes de agregar archivos, ejecuta esto dentro de la carpeta del examen, o clona el repo de GitHub:

   ```
   git init
   ```

4. **El script del modelo falla si se ejecuta de arriba a abajo.**
   - En la línea 204 llama a `fn_conteo_tablas()`, pero la función se crea en la línea 230.
   - El `CALL` de la línea 201 va antes de la importación de los CSV, así que en ese punto no carga nada.
   - En el README describí el orden correcto. Lo más limpio es mover las líneas 199–204 a después de la sección 8.
5. **El script tiene una contraseña real en la línea 251 (`lector_brechas`).** Tu documento dice que en Neon no usas restricción por IP. Cámbiala por un texto como `'<contraseña>'` antes de subir el archivo.
6. **El índice `idx_brechas_severidad_fecha_detecci` aparece en el plan de la consulta 1.** PostgreSQL leyó `brechas` usando `uq_brechas_cod`, porque así obtiene las filas ya ordenadas por

6. **El índice `idx_brechas_severidad_fecha_deteccion` no aparece en el plan de la consulta 1.** PostgreSQL leyó `brechas` usando `uq_brechas_codigo`, porque así obtiene las filas ya ordenadas por código para la agrupación, y con 800 filas eso le sale más barato. Esto no incumple nada, pero si el profe pregunta, esa es la explicación.

Si quieres, corrijo yo los puntos 2, 4 y 5 en los scripts: el encabezado, el orden de las secciones y la contraseña. El diagrama y el PDF de IA te tocan a ti. Para cerrar la entrega:

- Agrega a `jdrodas` como colaborador del repo.
- Envía el correo de notificación a juand.rodasm@upb.edu.co.

M↓ README.md Deshacer +80 −0 ›

¿Cómo está funcionando Claude en esta sesión? Mal Regular Bueno ×